

Automated Estimation of Link Quality for LoRa: A Remote Sensing Approach

Integrating Satellite Remote Sensing with Machine Learning Logic

Presented by : [HARSH T25102]

Guided by : [Dr.PRETI KUMARI]

LoRa Link Theory: Achieving Long Range with Low Power

The IoT Problem: Smart Cities need to connect thousands of sensors, but existing tech failed:

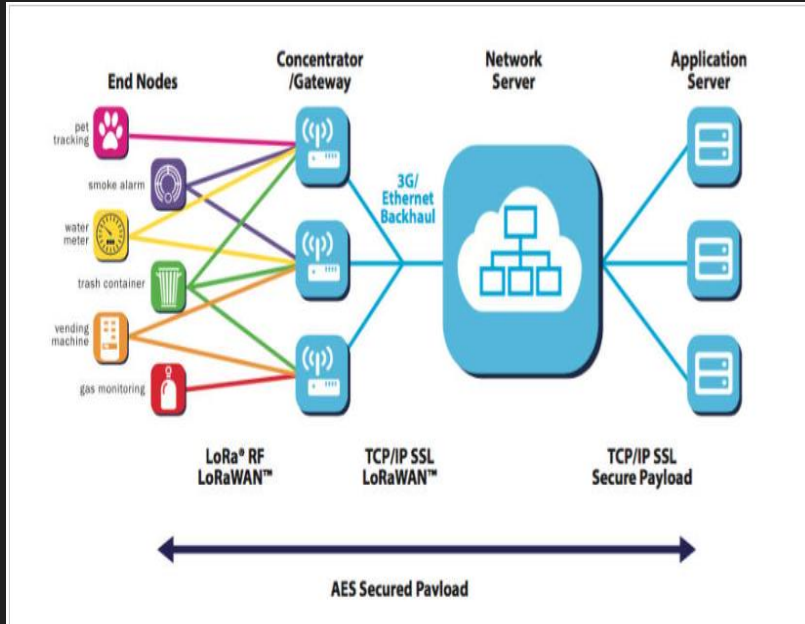
- **WiFi/Bluetooth:** Good speed, but **Range is too short** (< 50m).
- **Cellular (4G/5G):** Good range, but **consumes too much power** (batteries die in days).

The LoRa Solution:

- **Long Range:** Covers **2–15 km** in real urban settings.
- **Low Power:** Sensors last **10+ years** on a single battery.
- **High Sensitivity:** Works at **-140 dBm**, allowing it to penetrate basements and deep indoor areas where 4G fails.

Superior Sensitivity: LoRa receivers can detect signals as weak as **-140 dBm**, whereas WiFi stops at **-90 dBm**.

LoRaWAN System Architecture



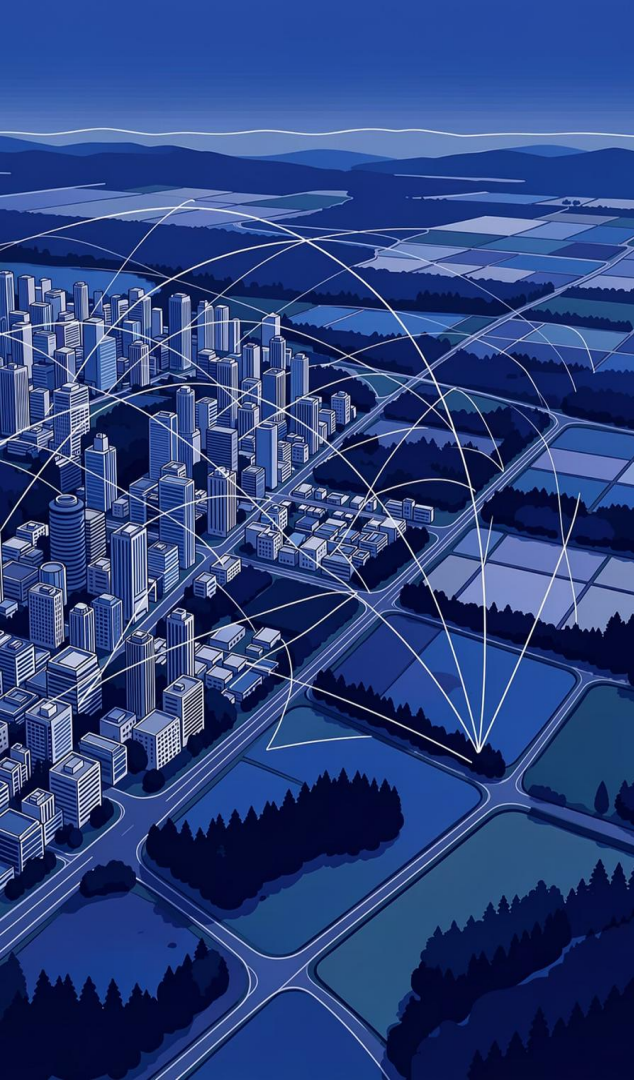
1. End Nodes (Devices): Low-power sensors that transmit data (e.g., smart meters, environmental sensors) using the LoRa modulation technique.

2. Gateways (Base Stations): Unlike cellular networks, nodes are **not** paired with a specific gateway. Instead, a node broadcasts to **all** nearby gateways, which simply forward the packet to the cloud

3. Network Server: It filters duplicate packets from different gateways, manages security keys, and routes data to the specific Application Server.

4. Application Server: The final destination where user data is processed and visualized

Reference : <https://www.3glteinfo.com/lora/lora-architecture/>



The Challenge: Why LoRa Coverage is Unpredictable

While LoRaWAN technology theoretically supports communication ranges up to **15 km** (in free space), real-world urban environments drastically reduce this range due to obstacles.

In dense cities, a single obstruction can block the signal entirely, cutting the effective range from kilometers down to just **200–500 meters**.

On-site measurements are impractical due to the large geographical areas involved.

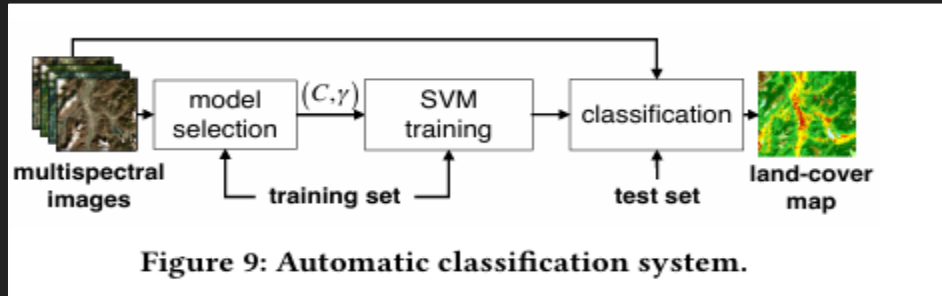
Paper Title : Automated Estimation of Link Quality for LoRa: A Remote Sensing Approach (IEEE IPSN)

1. **Satellite Data Acquisition:** Utilize **Sentinel-2** satellite imagery from the European Space Agency (ESA).

2. **Machine Learning & Modeling:**

a) **Classification:** A **Support Vector Machine (SVM)** algorithm is applied to the images to automatically classify land cover types (buildings, trees, and open fields).

b) **Site-Specific Modeling:** Instead of assuming the ground is flat, the system assigns a specific signal loss value to **every individual pixel**, creating a custom coverage map for the city.



3. **Path Loss Calculation:** The final model predicts signal strength by applying distinct **path loss formulas** that automatically adapt to **Urban** and **Semi-Urban** environments based on the detected obstacles.

Methodology: System Workflow

Step 1: Data Acquisition: We manually acquired **Sentinel-2 Level-2A imagery** for the target area (via Copernicus Open Access Hub), ensuring <10% cloud cover for data quality.

Step 2: Pre-processing (QGIS): Using **QGIS**, we combined bands (Red, Green, NIR) to generate a **False Color Map**. This highlights vegetation in bright red, helping us distinguish trees from urban buildings.

Step 3: SVM Classification: The **Support Vector Machine (SVM)** algorithm classifies every pixel into **5 categories**: Urban (Building), vegetation, Water, Soil, and Road.

Step 4: Land Cover Map Generation The SVM generates a classified **Land Cover Map**, labeling every pixel as either **Urban (Building)**, **Forest (Tree)**, **Water**, or **Open Land**.

Step 5: ESP Calculation Finally, we apply the **Okumura-Hata Model**. This calculates the **Estimated Signal Power (ESP)** in dBm for every point based on the land cover data.



False Color Map

Step-2 : Preparation in QGIS

- 1. Layer Stacking:** The individual Sentinel-2 spectral bands were imported into **QGIS** and combined into a **single multi-layer raster file**. This aligns all spectral data into one manageable file.
- 2. False Color Composite (FCC):** We generated a **False Color Map** (combining Near-Infrared, Red, Green) to visualize the area. This makes vegetation appear bright red, making it easier to distinguish from concrete buildings.
- 3. Training Data Creation** We manually created a vector layer of **Training Points**. We selected specific pixels on the map and labeled them according to their actual class (e.g., Urban, Forest, Water, Soil, Road).
- 4. CSV Data Export:** We extracted the pixel values from these points and exported them as a **.CSV file**. This file serves as the **Input Dataset** for training our SVM model in Python.

Step - 3 : SVM Classification (Python Implementation)

1. We loaded the QGIS CSV data and calculated two additional indices in Python to improve accuracy:

- **NDVI:** To highlight vegetation health.
- **NDWI:** To explicitly detect water bodies.

2. **Scaling the Numbers:** We used `StandardScaler` to shrink the large satellite values down to a standard range. This is necessary because SVM works best when all numbers are on the same small scale.

3. **Choosing the Model:** We selected the **SVM with RBF Kernel**.

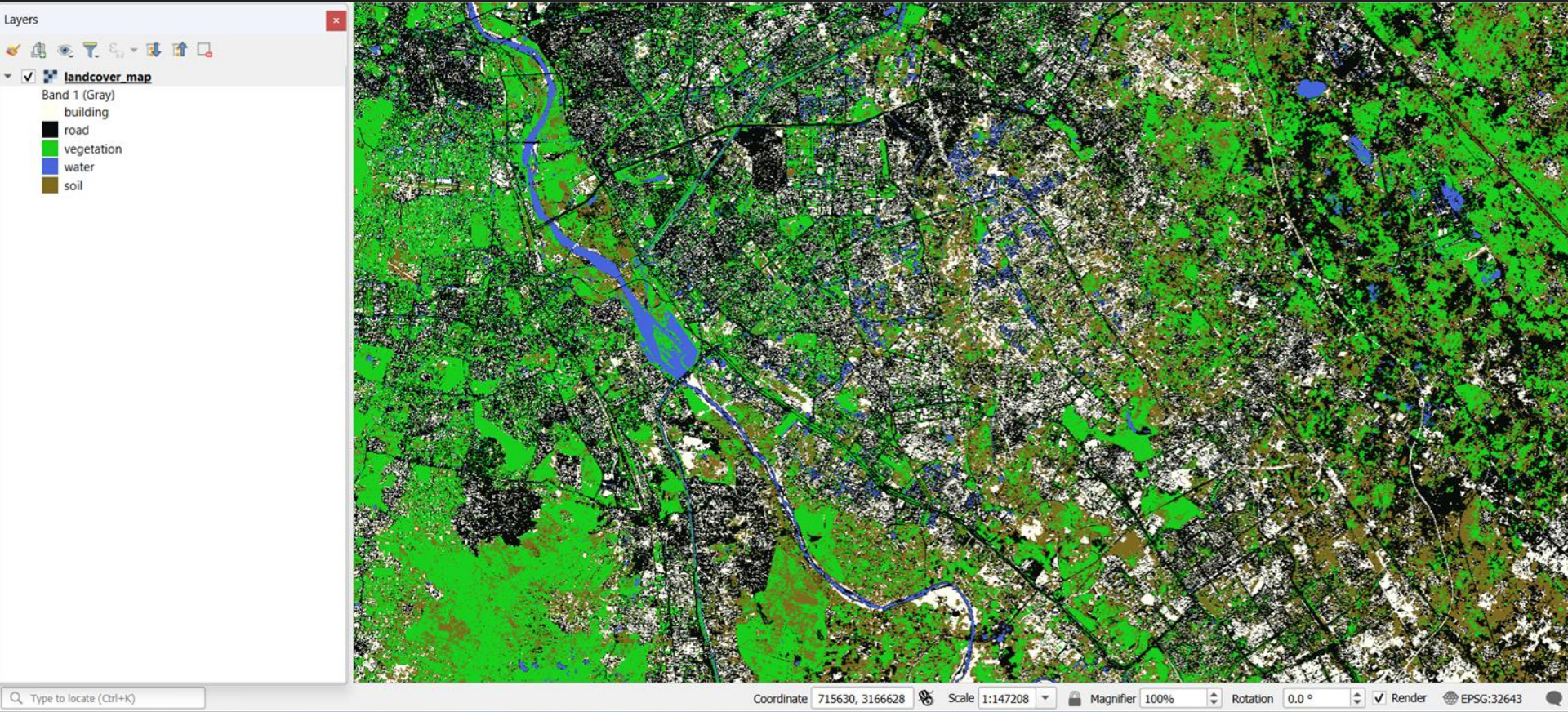
- *Reason:* Real-world boundaries (like a curved road or a forest edge) are not straight lines. The **RBF Kernel** allows the model to draw **curved boundaries** to fit the city map.

4. **Training Strategy:** We split the data: **70% for Training** and **30% for Testing**. This ensures we have enough data to check if the model is actually accurate before we use it.

Step-4 : Land Cover Classification

- 1. Automated Prediction:** We developed a custom Python script (`classify_full_image.py`) that automates the mapping process. It loads the trained **SVM Model** (`svm_model.pkl`) and applies it to fresh satellite data using the **joblib** library.
- 2. Raster Processing:** Using the **rasterio** library, the script reads the spectral bands (Blue, Green, Red, NIR) and calculates indices (NDVI, NDWI) for every single pixel in the city.
- 3. Class Segmentation:** The model predicts a specific class ID for each pixel, successfully segmenting the city into 5 categories:
 - **Class 0:** Buildings , **Class 1:** Roads, **Class 2:** Vegetation (Trees), **Class 3:** Water, **Class 4:** Soil
- 4. GeoTIFF Generation:** The script exports the predictions as a **GeoTIFF file** (`landcover_map.tif`). We visualized this file in QGIS to confirm that "Urban" pixels (Red) correctly align with the dense city center.

Step-4 : Result LandCover Classification



Step-5 :Path Loss Modeling & ESP Logic

1. Dynamic Path Analysis: We developed a Python script (`esp_prediction.py`) that calculates signal strength for any specific link. It uses the `skimage.draw.line` function to trace the exact path between the **Transmitter (Tx)** and **Gateway (Gw)**.

2. Obstacle Detection: The system checks the **Land Cover Map** along this path.

- If the path intersects **Buildings (Class 0)** or **Vegetation (Class 2)**, the environment is classified as **"Urban" (Non-Line-of-Sight)**.
- If the path is clear, it is classified as **"Suburban" (Line-of-Sight)**.

3. Okumura-Hata Implementation:

Based on the environment decision, the script dynamically selects the correct propagation formula:

- **Urban:** Uses the standard Okumura-Hata Urban model (`L_urban`).
- **Suburban:** Applies a correction factor to reduce path loss.

4. Final ESP Output: The system subtracts the calculated Path Loss from the Transmit Power (P_{tx}) to output the **Estimated Signal Power (ESP)** in dBm.

Results: Simulation & Validation

Parameter	Scenario A: City Center	Scenario B: Open Road
Distance	0.475 km	0.307km
Obstacle Detected?	YES (Building)	NO (Clear Path)
Environment Mode	Urban (NLOS)	Suburban (LOS)
Predicted Signal	-99.34 dBm (Weak)	-82.8 dBm (Strong)
Network Status	Critical Zone	Good Coverage

1. Simulation Setup:

We validated the model by simulating links between a Gateway and a Device at various locations to test the system's response to different land cover types.

2. Obstacle Detection Results:

- **Scenario A (City Center):** The algorithm detected buildings in the path and switched to **"Urban Mode" (NLOS)**.
- **Scenario B (Highway):** The path was clear, so the algorithm selected **"Suburban Mode" (LOS)**.

3. Signal Attenuation:

The results show a significant signal drop (**~17 dBm**) in the urban scenario due to building blockage, confirming that the model correctly accounts for physical obstacles.

C:\Users\Harsh\OneDrive\Desktop\LoR-Implement>C:\Users\Harsh\OneDrive\Desktop\LoR-Implement>python esp_prediction.py

Enter Deive location:

TX row: 6177

TX col: 4287

Enter Gateway location:

GW row: 6141

GW col: 4318

Enter device height (m): 2

Enter gateway height (m): 30

===== RESULT =====

Environment : URBAN

Distance (km): 0.475

Predicted ESP (dBm): -99.34

=====

Enter Deive location:

TX row: 6295

TX col: 4280

Enter Gateway location:

GW row: 6324

GW col: 4270

Enter device height (m): 2

Enter gateway height (m): 30

===== RESULT =====

Environment : SUBURBAN

Distance (km): 0.307

Predicted ESP (dBm): -82.8

=====

Conclusion :

- **Successfully Integrated AI & Physics:** We developed a hybrid model that combines Satellite Machine Learning (SVM) with Wireless Physics (Okumura-Hata). This allows for site-specific signal planning without expensive manual surveys.
- **Automated Land Classification:** Our Python automation achieved high accuracy in identifying Urban and Vegetation zones, which are the primary causes of signal blockage (NLOS).
- **Smart Signal Prediction:** Unlike basic distance-based calculators, our tool dynamically detects obstacles in the path, providing realistic signal estimates (-100 dBm vs -82 dBm) for better network planning.

Future Scope :

- **3D City Modeling:** Integrate Digital Surface Models (DSM) to calculate signal diffraction over specific building heights, not just general "Urban" areas.
- **Web-Based Interface:** Convert the Python script into a user-friendly Web Application (using Streamlit or Flask) so network engineers can use it easily.